

 前文提到人工智能的优化为如下公式：

期望风险最小化（理论）≈ 经验风险最小化（参数最优，或者说减少训练误差）+ 降低模型复杂度（限制参数取值范围，或者说减少泛化误差）

经验风险

模型复杂度

$$\arg \min_{\theta} \frac{1}{N} \sum_{n=1}^N \mathcal{L}(y^{(n)}, f(\mathbf{x}^{(n)}, \theta)) + \lambda \ell_p(\theta)$$

知乎 @P9工作法

而 [一文搞懂经验风险最小化](#) 已经介绍了如何通过优化[梯度下降算法](#)来降低经验风险。这里将接着介绍如何降低模型复杂度。主要包含，参数初始化，L1&L2正则，干扰优化过程三个方面。

参数初始化

参数初始化对于神经网络非常重要，最底层的原因就是神经网络都是非凸问题，不同的参数初始化可能会让模型找到不同的局部最优解。关于参数初始化可以想到几个方法：

- 一是用预训练好的模型来做初始化，比如用预训练好的模型LLMA来做微调。
- 二是基于固定值初始化，但是不能初始化为0，如果初始化为0，那么不管你输入的是什么数据，和权重乘积都变成0，梯度也变成0，这就失去了意义。
- 三是随机值初始化，比如使得权重符合高斯分布，或者均匀分布等等。这里也有很多工具来做，理解到这里就可以了。

L1&L2(增加约束限制参数的范围)

降低模型复杂度最主要的是给模型加上一些[正则化项](#)，常见的正则化项包括 L1 和 L2 正则化，它们分别对应于模型参数的 L1 [范数](#)和 L2 范数。降低模型复杂度是限制模型参数的取值，提升模型的泛化能力。

以逻辑回归来说，损失函数如下：

$$L(\theta) = -\frac{1}{n} \sum_{i=1}^n [y_i \log(\hat{y}_i) + (1 - y_i) \log(1 - \hat{y}_i)]$$

如果加上正则化，即表达为：

$$L(\theta) = -\frac{1}{n} \sum_{i=1}^n [y_i \log(\hat{y}_i) + (1 - y_i) \log(1 - \hat{y}_i)] + \frac{\lambda}{2} \|\theta\|_2^2$$

若加上的是L1正则，即表达为：

$$L(\theta) = -\frac{1}{n} \sum_{i=1}^n [y_i \log(\hat{y}_i) + (1 - y_i) \log(1 - \hat{y}_i)] + \lambda \|\theta\|_1$$

可以看到，其实就是在损失函数后面加上一段，这就是L1、L2范数。从这个公式可以看出来，加上了正则项后，会使得参数的值也变得小一些。

来个直白的理解，比如我们模拟出来的函数是：

$$y = W_1 x_1 + W_2 x_2 + W_3 x_3 + W_4 x_4 + \dots + W_n x_n$$

这样的一个小阶函数可能就比较完美地拟合数据了，但这并不是我们想要的，因为有可能 W_n 这个参数贡献特别大，把曲线搞得特别弯，这对训练数据拟合很好，但是对于未来数据预测就可能不准。那如果有一个方法能够限定这些参数趋近于零（或者是直接置为0，比如drop out就是随机让某些神经元变成0），就可能会好一些。比如使得函数变得类似

$y = W_1 x_1 + W_2 x_2 + W_3 x_3 + W_4 x_4 + \dots + W_n x_n$ 这样的效果就可以了（反过来看也是一样的道理，如果某个 W 过大，会使得轻微的数据改变就会导致结果有很大差异，模型的表现就不稳定）。这就是加入正则项来限制参数范围的意义。

这里也提一下，正则属于降低模型复杂度的范畴（因为神经元很多，这个函数就是会很复杂，但是神经元不多，层数不够也无法更好地拟合，所以需要权衡），可以看成是降低泛化误差。而前面讲的经验风险最小化，其实是降低训练误差，就是找到更优秀的参数取值。

L1范数

看到上面的表达式，其实可以发现，加入正则项目后，就是利用[拉格朗日法](#)求解参数。L1表达式如下：

$$\text{Regularization}(\mathbf{w}) = \|\mathbf{w}\|_1 = |w_1| + |w_2| + \dots + |w_m|$$

就是加入w的绝对值。从前面的拉格朗日乘子法+的角度来说，就是对参数w加入了一些限制条件，在这些限制条件下去求得参数的解法。对应的拉格朗日函数+ (关于拉格朗日相关将单独再开文章来总结)如下：

$$L(\boldsymbol{w}, \lambda) = J(\boldsymbol{w}) + \lambda(\|\boldsymbol{w}\|_1 - C)$$

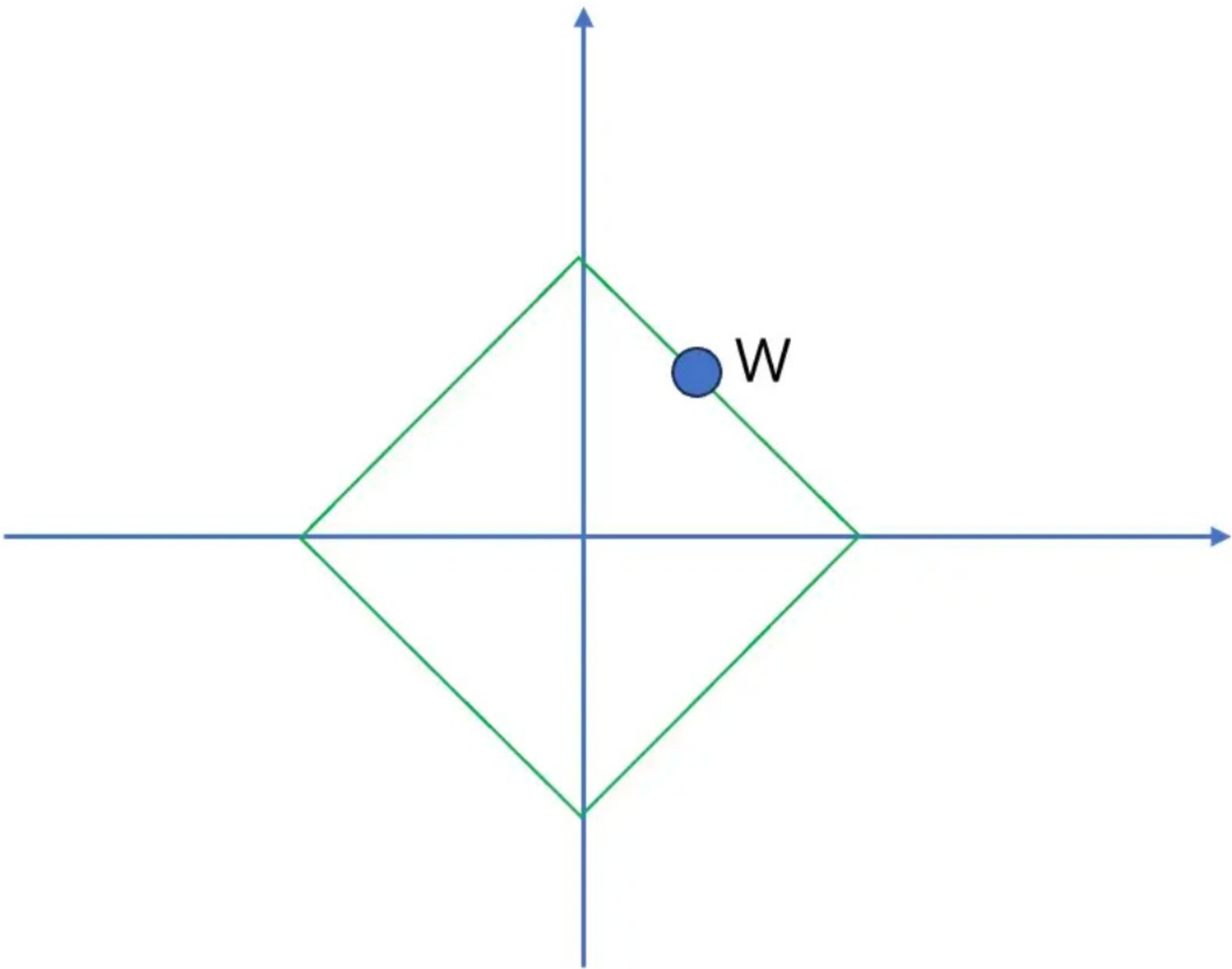
$$\min_{\boldsymbol{w}} \max_{\lambda} L(\boldsymbol{w}, \lambda)$$

$$\text{s.t. } \lambda \geq 0$$

以两个参数为例，那么L1范数+相同的点用图形画出来就是这样一个菱形

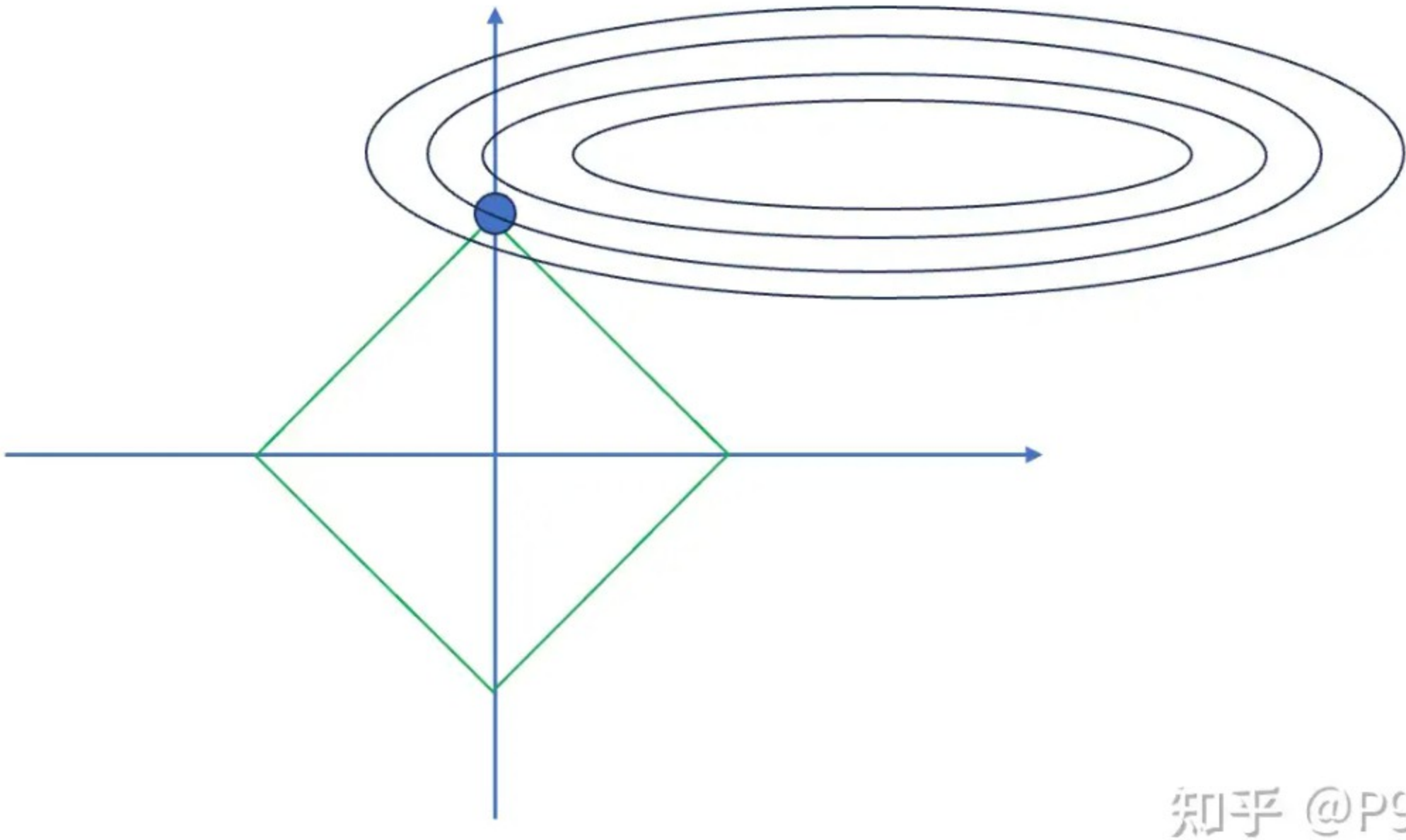
$$|\boldsymbol{W}|_1 = |w_1| + |w_2| + \cdots + |w_m|$$

$$|\boldsymbol{W}|_1 = |w_1| + |w_2| + \cdots + |w_m|$$

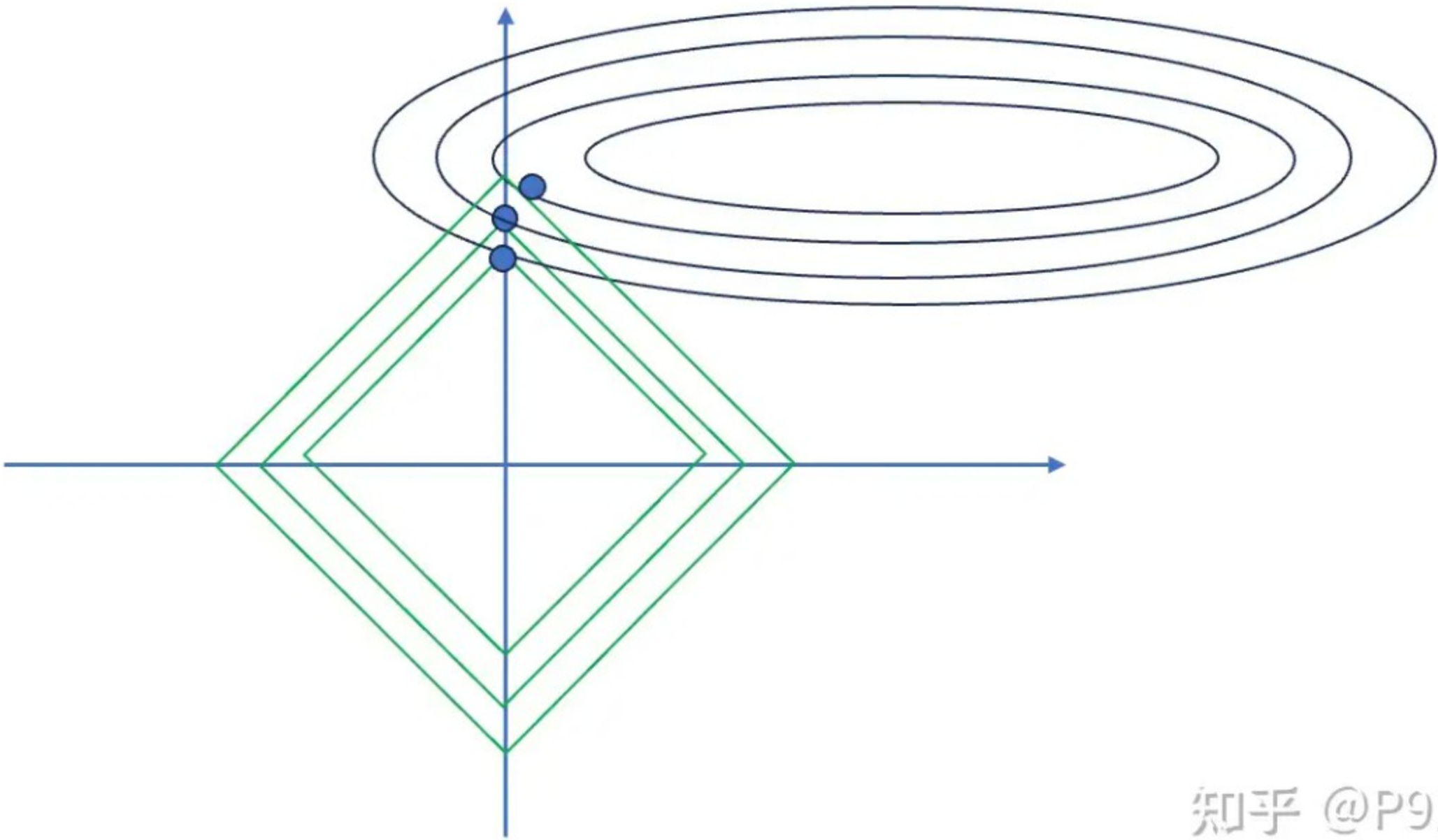


知乎 @P9工作法

如果把损失函数的等线画出来，与L1范数结合在一起就可以看到有如下的解。



而 λ 参数就是调节菱形的范围，焦点就是最优解，可以看到L1最容易产生稀疏解（在坐标轴上，那么另外一个维度取值就是0）



L2范数

L1表达式如下

$$\text{Regularization}(\boldsymbol{w}) = \|\boldsymbol{w}\| = \sqrt{w_1^2 + w_2^2 + \cdots + w_m^2}$$

就是加入w的平方，来求解参数。对应的拉格朗日函数如下：

$$L(\boldsymbol{w}, \lambda) = J(\boldsymbol{w}) + \lambda(\|\boldsymbol{w}\|_2 - C)$$

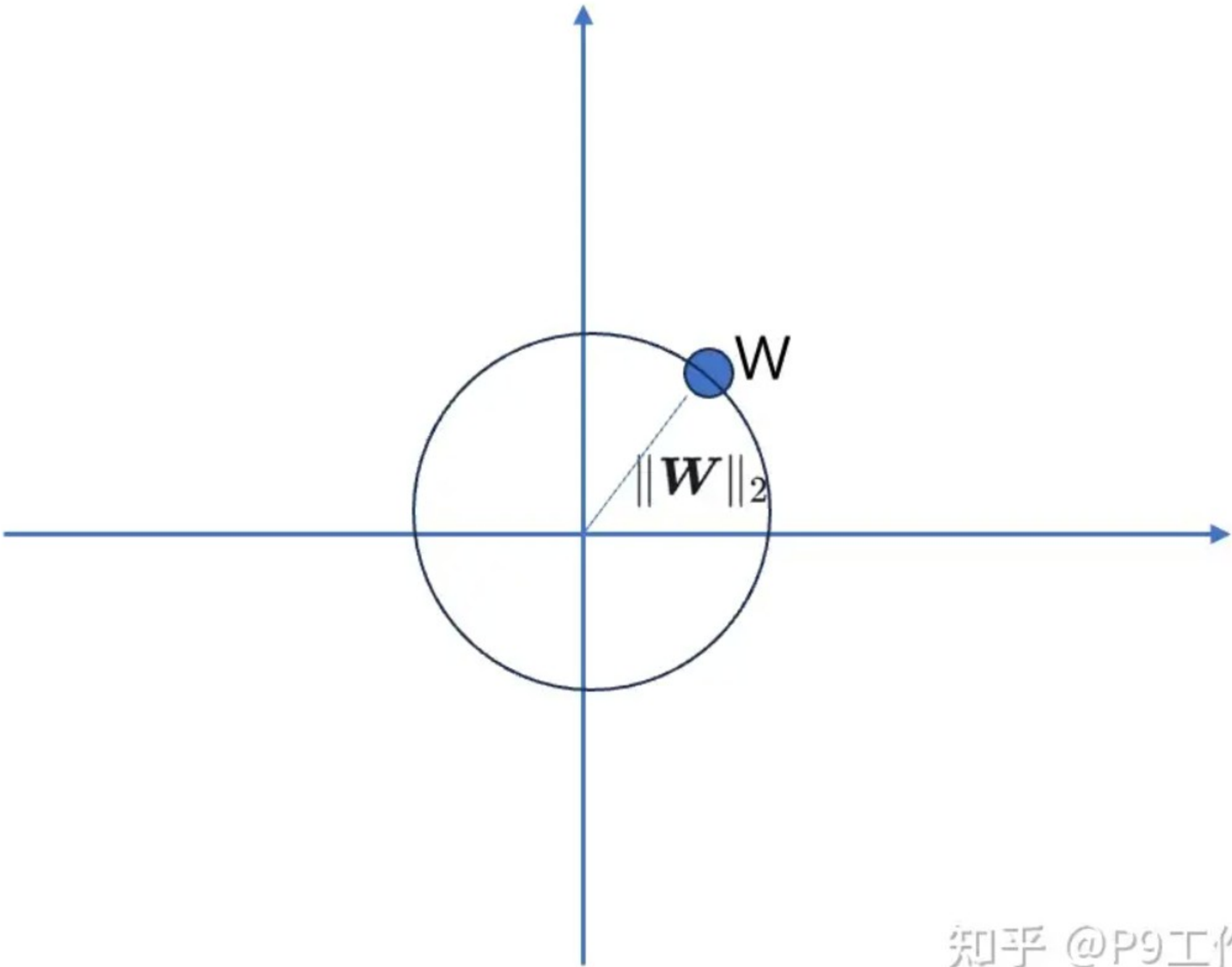
$$\min_{\boldsymbol{w}} \max_{\lambda} L(\boldsymbol{w}, \lambda)$$

$$\text{s.t. } \lambda \geq 0$$

L2范数相同的点用图形画出来就是这样一个圆形（二维的情况）

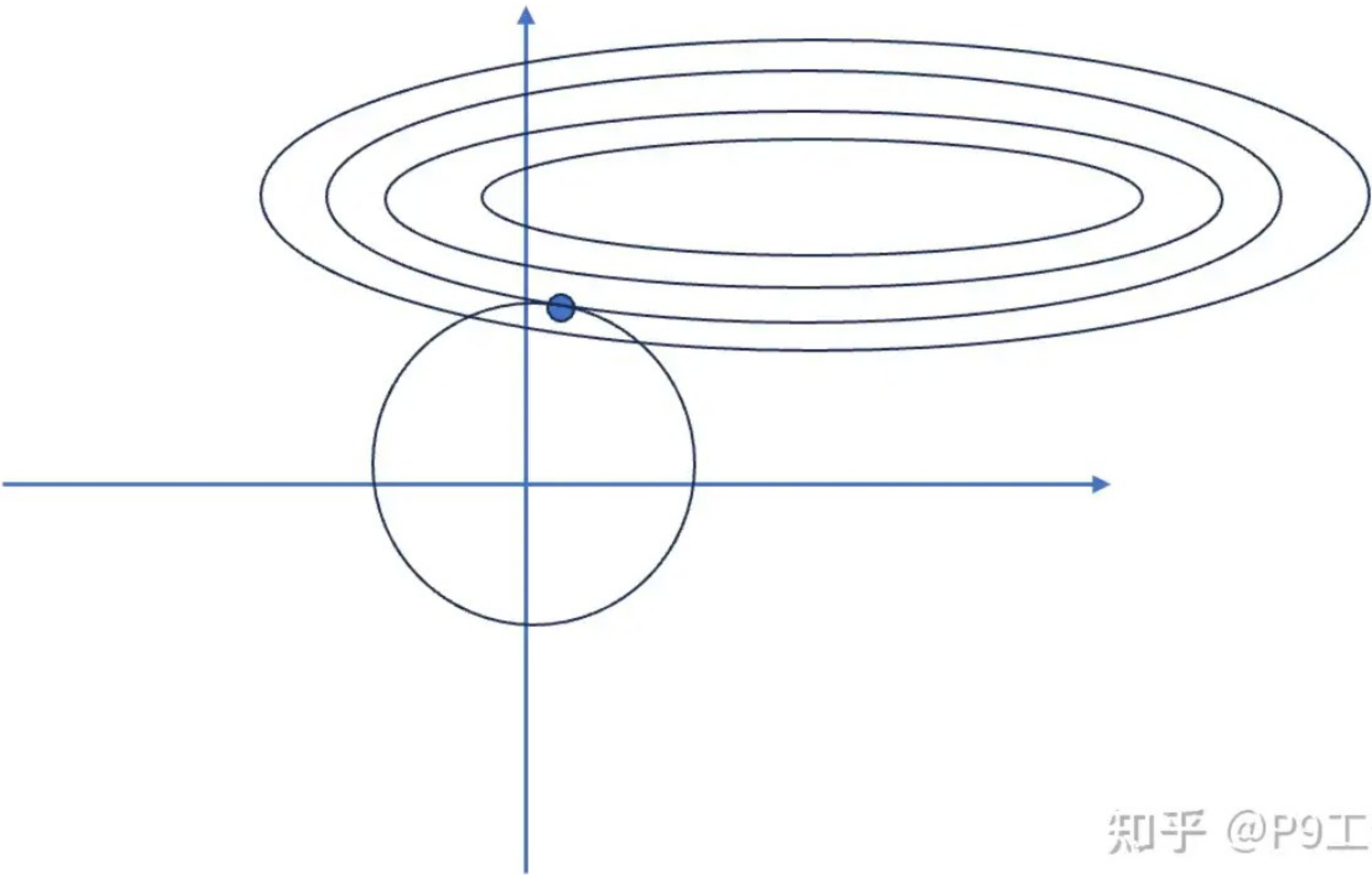
$$\|\boldsymbol{W}\|_2 = \sqrt{w_1^2 + w_2^2 + \cdots + w_m^2}$$

$$\|\boldsymbol{W}\|_2 = \sqrt{w_1^2 + w_2^2 + \cdots + w_m^2}$$



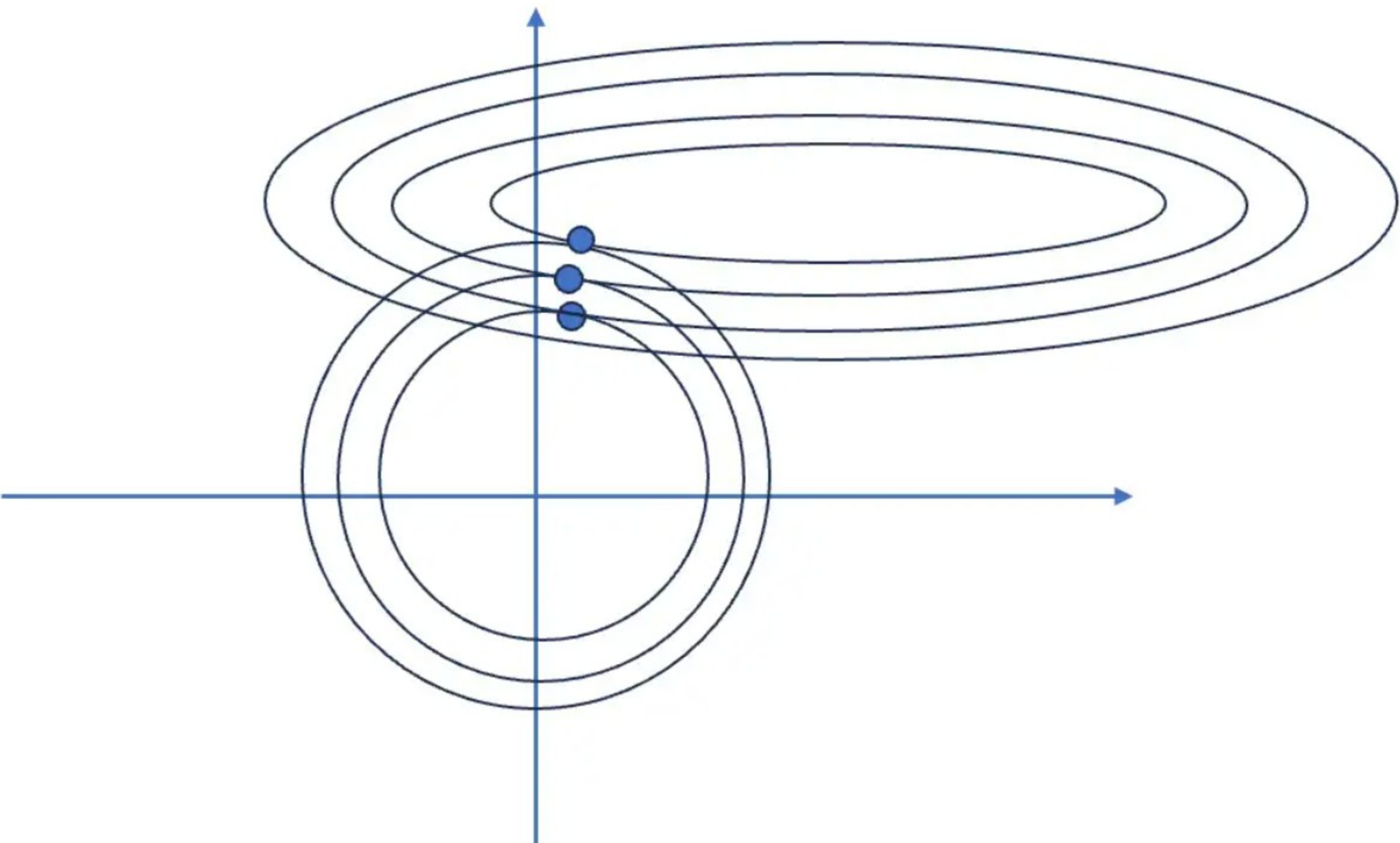
知乎 @P9工作法

如果把损失函数的等线画出来，与L1范数结合在一起就可以看到有如下的解。



知乎 @P9工作法

而 λ 参数就是调节菱形的范围，焦点就是最优解，可以看到L1不容易产生稀疏解，也就是横纵轴上都有取值。



所以比较下来，L1更容易区分出来特征，比如判断人和猴子通过尾巴这个特征就判段出来了。L2是每个特征都需要，只是判断值有差异，比如判断人和猴子，要精细化去看毛发多少这个特征。

干扰优化过程

early stop

在训练过程中，通过监控验证集的表现，设置停止条件，如连续几个epoch没有提升或达到某个阈值就提前停止训练，以防止模型过拟合，节省时间和计算资源。

dropout

通过随机丢弃一部分神经元，使模型在训练过程中不会过度依赖某些特定的特征，从而提高模型的泛化能力。同时也增加模型的鲁棒性⁺：使模型更加健壮，减少对单一路径的依赖，提高模型的稳定性和可靠性。

在每个训练批次中，随机选择一部分神经元（及其连接）暂时不参与前向传播和反向传播⁺。被丢弃的神经元的输出值设为0，相应的权重梯度也不更新。

从现实团队的例子来讲，就是去除对某些单点的依赖（只是这里换成了部分神经元链接），对整个组织稳定性是更好的。

超参数优化

超参数优化⁺是神经网络优化的最优一趴，很多人说算法工程师是调参工程师就是指这部分的参数。常常设计的超参数有如下表

超参数类型	参数名
网络机构	层数
每层神经元个数	
激活函数类型	
优化参数	小样本数量
学习率	
梯度修正方法	
正则化系数 ⁺	

但是超参数优化没有通用的方法，评估代价是很高的。常用的有：网格搜索，随机搜索，[贝叶斯优化⁺](#)，动态资源分配，神经架构搜索等。

小结一下：神经网络的优化分文了上下两文来阐述，神经网络优化分为三大部分：

- 经验风险最小化，详见 [一文搞懂经验风险最小化](#) ， 主要是如何优化梯度下降算法去寻求最优解。
- 降低模型复杂度，主要是L1&L2正则化。
- 超参数优化，即调整哪些人为设定（无法通过模型学到的参数）的参数使得模型最优。